DermaScan Project Architecture & Interview Preparation Guide

Version: 1.0 **Prepared for:** Technical/Project Interview Readiness **Projects:** `skin-disease-ui` (frontend) + `skin-disease-backend` (API/ML) **Last Updated:** 2026-10-28

Table of Contents

1. Project Overview
2. High-Level System Architecture
3. Repository Trees
4. Local Setup and Run Instructions
5. Frontend Architecture Deep Dive
6. Backend Architecture Deep Dive
7. Complete Image Prediction Flow
8. Key Terminology
9. Interview Questions (with concise answers)
10. Expected Cross-Questions (with concise answers)
11. Known Limitations and Production Improvements
12. Demo Checklist
13. Five-Day Preparation Plan
14. 60-Second Self-Introduction
15. Safety and Medical Caveats

1) Project Overview

DermaScan is an AI-assisted skin lesion analysis platform:

• **Frontend (**`skin-disease-ui`**)**: React + Vite single-page app for image upload, prediction display, explainability views, history, and role-aware dashboards.

• **Backend (**`skin-disease-backend`**)**: Flask API serving prediction endpoints, Grad-CAM explainability, community insights, verification/retraining workflows, and MySQL persistence.

• **ML core**: TensorFlow/Keras ResNet-based classifier for HAM10000-style skin lesion classes.

• **Core value proposition**: Fast, explainable preliminary risk screening support with a strong warning that outputs are **not** medical diagnosis.

2) High-Level System Architecture

```
[Browser UI: React + Vite]
  ■■ Upload image (multipart/form-data)
  ■■ Call /predict and supporting APIs
  ■■ Show diagnosis + confidence + XAI heatmaps
  ■■ Store user/session/history cache in localStorage
  ■■ Display fallback/mock data on non-critical API failures

          ■ HTTP (proxy in dev, direct API base in prod)
          ▼
[Flask API Layer]
  ■■ Input validation and error handling
  ■■ Image decode + preprocessing
  ■■ TensorFlow ResNet inference
  ■■ Class mapping + confidence scoring
  ■■ Grad-CAM generation + overlays
  ■■ Business endpoints (treatment, insights, retraining, verification)
  ■■ JSON responses

          ■
          ■■■■■■■■■■■ [TensorFlow Model: models/resnet_model.h5]
          ■
          ■■■■■■■■■■■ [MySQL]
                      ■■ analysis_history
                      ■■ doctor_verifications
                      ■■ community_insights
                      ■■ preventive_care
                      ■■ preventive_care_recommendations
```

3) Repository Trees

3.1 `skin-disease-ui` (frontend)

```
.
■■■ .github/workflows/deploy-pages.yml
■■■ README.md
■■■ package.json
■■■ vite.config.js
■■■ public/
■   ■■■ ai-scan.json
■   ■■■ vite.svg
■■■ src/
    ■■■ App.jsx
    ■■■ api.js
    ■■■ NotificationService.js
    ■■■ index.jsx
    ■■■ index.css
    ■■■ components/
        ■■■ Auth.jsx
        ■■■ CommunityInsights.jsx
        ■■■ DermatologistFinder.jsx
        ■■■ Disclaimer.jsx
        ■■■ Header.jsx
        ■■■ Home.jsx
        ■■■ Results.jsx
        ■■■ UploadSection.jsx
        ■■■ UserProfile.jsx
```

```
    ■■■ XAIExplanation.jsx
    ■■■ ...
```

3.2 `skin-disease-backend` (backend)

```
.
■■■ app.py
■■■ requirements.txt
■■■ Dockerfile
■■■ gunicorn.conf.py
■■■ render.yaml
■■■ test_api.py
■■■ database/
■   ■■■ db_connection.py
■■■ models/
■   ■■■ resnet_model.h5
■■■ utils/
■   ■■■ image_processor.py
■   ■■■ model_loader.py
■   ■■■ xai_explainer.py
■■■ uploads/
```

4) Local Setup and Run Instructions

4.1 Frontend (`skin-disease-ui`)

```
cd skin-disease-ui
npm install
npm run dev
```

• Vite dev server typically runs on `http://localhost:5173`.

• Dev proxy (`vite.config.js`) forwards `/api`, `/predict`, `/suggest_treatment`, `/health` to backend `http://localhost:5001`.

4.2 Backend (`skin-disease-backend`)

Recommended based on project pinning and Docker base image:

```
cd skin-disease-backend
python -m venv venv
source venv/bin/activate  # Windows: .\\venv\\Scripts\\Activate.ps1
pip install --upgrade pip setuptools wheel
pip install -r requirements.txt
python app.py
```

Backend target URL:

```
http://localhost:5001
```

Quick health check:

```
http://localhost:5001/health
```

4.3 Practical dependency note from troubleshooting

When pinned dependency conflicts appear (for example `mysql-connector-python` vs `tensorflow` protobuf requirements), either:

1. Align package pins in `requirements.txt`, or
2. Use the project's Docker workflow (`python:3.10-slim`) as the canonical runtime.

5) Frontend Architecture Deep Dive

5.1 Core stack

Area	Choice	Why it matters
Framework	React 19	Component-driven UI + hooks
Bundler	Vite 7	Fast local dev + optimized builds
Routing	React Router	Route-based navigation + protected views
HTTP	Axios	Interceptors, timeout, retry patterns
UI	Tailwind + MUI + Framer Motion	Rapid styling + polished UX

5.2 App shell and routing (`src/App.jsx`)

• Uses `BrowserRouter`, `Routes`, `Route` with role-aware `ProtectedRoute` guards.

• Main routes include `/analysis`, `/results`, `/xai-explanation`, `/community`, `/history`, `/profile`, `/admin`, `/doctor`.

• Includes error boundary, loading states, and lazy loading for heavy components.

5.3 State and effects

Main app state includes:

• `user`

• `currentAnalysis`

• `analysisHistory`

• `retrainingStatus`

• `modelPerformance`

• `communityInsights`

• `preventionData`

React effects manage:

• App bootstrap from `localStorage`

• Polling retraining/model metrics

• History persistence with quota-aware cleanup

• Community insights refresh

5.4 Axios API layer (`src/api.js`)

• Centralized `axios.create` with base URL and timeout.

• Request interceptor injects token and metadata.

• Response interceptor adds retry logic for transient failures.

• In-memory API cache (`Map`) with namespace + TTL.

• Fallback mock generators for resilience when selected endpoints fail.

5.5 Proxy, CORS, and environment behavior

• Vite dev proxy sends API calls to backend.

• Backend enables CORS (`CORS(app)`), allowing cross-origin frontend calls.

• `VITE_API_BASE_URL` can override API host in frontend runtime.

5.6 Caching and persistence strategy

• `localStorage` used for user session/history and some cached API payloads.

• Custom compression trims large fields before storing history.

• Quota checks trigger cleanup policy to reduce app breakage.

5.7 Mock fallback behavior

If APIs fail for non-critical paths (community/preventive/heatmap/history/XAI), frontend can still render with mock data so demo/interview flow is not blocked.

6) Backend Architecture Deep Dive

6.1 Core stack

Layer	Tech
Web framework	Flask
CORS	flask-cors
ML runtime	TensorFlow/Keras + ResNet model
Image handling	Pillow + OpenCV + NumPy

Layer	Tech
Database	MySQL (`mysql-connector-python`)
Prod serving	Gunicorn
Container/deploy	Docker + Render

6.2 API routes and request/response flow (selected)

Route	Method	Purpose
`/`	GET	Health/service availability
`/predict`	POST	Main inference + XAI payload
`/suggest_treatment`	POST	Treatment recommendation flow
`/api/community-insights`	GET	Aggregated trend data
`/api/preventive-care`	GET	Prevention guidance
`/api/verify-diagnosis`	POST	Doctor feedback capture
`/api/model-performance`	GET	Accuracy and retraining signals
`/api/retrain-model`	POST	Trigger retraining process
`/api/retraining-status`	GET	Retraining progress

6.3 Image upload and preprocessing

1. Accept file under `image` or `file` key.
2. Save to `uploads/` with secure filename.
3. Load image and resize to model input dimensions.
4. Normalize/preprocess using TensorFlow ResNet preprocessing.
5. Validate shape against model expectation.

6.4 Inference and post-processing

• Load model from `models/resnet_model.h5`.
• Run `model.predict(...)`.
• Map index to lesion class labels.
• Compute confidence and top-k prediction list.
• Determine cancer-risk flag through class-to-risk mapping.

6.5 Grad-CAM / XAI flow

• Detect a suitable convolutional layer.
• Build gradient model.
• Compute gradients against target class score.

• Generate normalized heatmap.

• Create superimposed image overlay.

• Return base64 images + explanation metadata for frontend rendering.

6.6 MySQL data model (from `app.py` table initialization)

Table	Intent
`doctor_verifications`	Store dermatologist verification feedback
`analysis_history`	Store predictions and metadata
`dermatologists`	Directory/lookup records
`community_insights`	Daily aggregate counters and disease breakdown
`preventive_care`	Disease-specific prevention records
`preventive_care_recommendations`	Skin-type/concerning-pattern recommendations

6.7 Error handling and resilience

• Extensive try/except around database/model paths.

• Fallback JSON responses for some failed database operations.

• Validation failures return `400`; server/internal issues return `500`.

6.8 Deployment model

• **Dockerfile** uses `python:3.10-slim` and installs OpenCV-related system libs.

• **Gunicorn** runs single worker (`workers=1`) tuned for TensorFlow/container constraints.

• **Render** `healthCheckPath: /health` and runtime env var wiring.

7) Complete Image Prediction Flow

```
[User selects image in UploadSection]
  -> validate extension + size
  -> create FormData
  -> POST /predict

Backend /predict:
  -> validate request payload
  -> save upload file
  -> load & preprocess image
  -> run ResNet inference
  -> derive predicted class + confidence + risk status
  -> compute top predictions
  -> attempt Grad-CAM generation
  -> build JSON response with diagnosis + explanations

Frontend:
  -> render Results page
  -> show confidence bars + status tags
```

```
-> show XAI explanation and heatmap (if present)
-> request treatment/prevention data
-> optionally store in analysis history
```

8) Key Terminology

Term	Meaning
HAM10000	Public dermatoscopic skin lesion dataset family often used in classification tasks
ResNet	Residual neural network architecture used here for lesion classification
Confidence score	Model-reported probability-like certainty for predicted class
Grad-CAM	Visual explanation method highlighting image regions influencing prediction
XAI	Explainable AI, techniques to make model behavior interpretable
CORS	Cross-Origin Resource Sharing policy for browser API access
Retraining	Updating model parameters using new/verified examples
Top-k predictions	Highest probability candidate classes returned alongside primary prediction

9) Interview Questions (with concise answers)

Q1. Why React + Vite for this project?

A: React gives componentized, scalable UI architecture; Vite gives faster startup/HMR and straightforward env/proxy setup.

Q2. Why centralized API layer?

A: It standardizes retries, timeouts, auth header injection, and error normalization while keeping components simpler.

Q3. How do you ensure UX when backend is unstable?

A: Non-critical endpoints support cached/mock fallbacks, preserving usable UI and demos while logging failures.

Q4. How does explainability work?

A: Backend computes Grad-CAM maps and confidence metadata; frontend renders overlays and explanation summaries for transparency.

Q5. How is data persistence handled in frontend?

A: `localStorage` stores user/history/cache with size checks and cleanup strategy to reduce quota failures.

Q6. How is model monitoring integrated?

A: Doctor verification data is collected and summarized to infer accuracy trends and retraining recommendations.

Q7. Why single Gunicorn worker?

A: TensorFlow memory footprint and CPU-bound inference make conservative worker count safer for small deployments.

Q8. What is your biggest architectural tradeoff?

A: High functionality in monolithic `app.py` improves speed of iteration but reduces modularity; refactoring into service modules is a next step.

10) Expected Cross-Questions (with concise answers)

CQ1. How do you handle invalid image uploads?

• Validate MIME/size client-side.

• Re-validate server-side (`no image`, empty filename, load failures).

• Return user-friendly 4xx errors.

CQ2. What if Grad-CAM fails?

• Backend catches exception and still returns diagnosis with fallback explanation fields.

• Frontend can still render core diagnostic section.

CQ3. How do you prevent over-trusting AI output?

• Persistent disclaimer UX + explicit wording that output is informational only.

• Encourage dermatologist verification workflows.

CQ4. How do you secure secrets?

• Use env vars for DB credentials in deployment.

• Avoid committing `.env` values.

• Keep sensitive values outside frontend bundle.

CQ5. How would you scale this architecture?

• Separate inference service from API gateway.

• Add async queue for heavy jobs.

• Externalize object storage and caching (Redis/CDN).

• Add observability and rate limiting.

11) Known Limitations and Production Improvements

Current Limitation	Production Improvement
Monolithic backend file	Split into blueprints/services/repositories
Basic auth/session handling	JWT/OAuth + robust RBAC + refresh strategy
Limited test coverage around ML/API integration	Add contract/integration and smoke tests in CI
Potential dependency pin conflicts	Maintain compatibility matrix + lockfile process
In-memory frontend cache only	Add shared cache layer and better invalidation
Local uploads folder	Move to cloud object storage with signed URLs
No full PHI-grade compliance workflow	Implement full security/compliance controls and audit trails

12) Demo Checklist

• [ ] Backend running and `/health` reachable.

• [ ] Frontend running and connected to backend.

• [ ] Upload at least one valid image.

• [ ] Show prediction, confidence, and risk flag.

• [ ] Open XAI explanation/heatmap view.

• [ ] Demonstrate community insights or fallback behavior.

• [ ] Show disclaimer and say “AI output is not final diagnosis”.

• [ ] Mention one limitation and one future improvement.

13) Five-Day Preparation Plan

Day	Focus	Output
Day 1	Architecture map + repos walkthrough	10-min whiteboard explanation
Day 2	Frontend deep dive (routing/state/api/cache)	Clear story for React decisions
Day 3	Backend + ML flow + DB + deployment	End-to-end request narrative
Day 4	Mock interview Q&A + cross-questions	Concise, repeatable answers
Day 5	Full demo rehearsal + intro pitch	Interview-ready practical run

14) 60-Second Self-Introduction

“Hi, I’m a developer who built and integrated DermaScan, an AI-assisted skin analysis platform. I worked across the frontend in React/Vite and backend in Flask with TensorFlow inference. My focus was building an end-to-end flow: image upload, model prediction, explainability with Grad-CAM, and role-based workflows for verification and retraining signals. I also designed fallback mechanisms and local caching so the app remains usable even when some APIs are unavailable. I’m comfortable discussing architecture tradeoffs, deployment with Docker/Gunicorn/Render, and how to evolve this

prototype into a production-grade, medically responsible system.”

15) Safety and Medical Caveats (Must Say in Interview)

1. DermaScan provides **AI-assisted informational output only**.
2. It is **not a confirmed medical diagnosis**.
3. Users must consult qualified dermatologists/healthcare professionals for diagnosis and treatment.
4. Emergency or rapidly changing symptoms require immediate professional care.

Appendix: Helpful Command Snippets

Frontend

```
npm install
npm run dev
npm run build
```

Backend

```
pip install -r requirements.txt
python app.py
```

Quick endpoint test

```
curl -X POST http://localhost:5001/predict \\
  -F "file=@/path/to/image.jpg"
```